

Lulo V2

Smart Contract Security Assessment

December 2024

Prepared for:

Lulo Labs

Prepared by:

Offside Labs

Yao Li

Sirius Xie





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Max Exposure Check May Fail Due to No Valid Deposit Solution	6
4.2	Redundant Checks May Lead to Solend Withdrawal Failure	7
4.3	Improper Obligation Initialization Check May Lead to Initialization Failures . .	8
4.4	Inconsistent Pending Withdrawal Amount Counting for Weight Calculation . .	9
4.5	Possible Precision Loss When Calculating Referral Fee Share	10
4.6	Precision in Referral Rewards Calculation May Leave Dust in UserReferrer . . .	12
4.7	Slight Decrease of External Protocol TVL Can Cause Protocol DoS	13
4.8	Informational and Undetermined Issues	14
5	Disclaimer	18



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *Lulo V2* smart contracts, starting on Dec 05, 2024, and concluding on Dec 20, 2024.

Project Overview

Lulo V2 is a DeFi insurance product offering two deposit types: PUSD and LUSD. PUSD earns dynamic yield with insurance against smart contract risks and limited exposure to DApps, while LUSD covers PUSD losses in the event of a hack. LUSD depositors earn yield from DApps and a share of PUSD's interest but take on the full risk of covering any PUSD losses. The system is designed to maximize returns while protecting PUSD depositors through exposure limits and interest redistribution.

Audit Scope

The assessment scope contains mainly the smart contracts of the flexlend program for the *Lulo V2* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Lulo V2
 - Codebase: <https://github.com/lulo-labs/flexlend>
 - Commit Hash: 8250edf7ff0d45cf19c7d999e5545a737b6f916c

We listed the files we have audited below:

- Lulo V2
 - programs/flexlend/src/instructions/pool/*.rs
 - programs/flexlend/src/state/pool/*.rs
 - programs/flexlend/src/state/user.rs
 - programs/flexlend/src/utils/math.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 2 medium issues



- 5 low issues
- 6 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Max Exposure Check May Fail Due to No Valid Deposit Solution	Medium	Fixed
02	Redundant Checks May Lead to Solend Withdrawal Failure	Medium	Fixed
03	Improper Obligation Initialization Check May Lead to Initialization Failures	Low	Fixed
04	Inconsistent Pending Withdrawal Amount Counting for Weight Calculation	Low	Fixed
05	Possible Precision Loss When Calculating Referral Fee Share	Low	Fixed
06	Precision in Referral Rewards Calculation May Leave Dust in UserReferrer	Low	Fixed
07	Slight Decrease of External Protocol TVL Can Cause Protocol DoS	Low	Acknowledged
08	Missing security_check in pool_refresh_kamino_alt IX	Informational	Fixed
09	Missing Range Checks When Updating Pool Parameters	Informational	Fixed
10	Precision Loss Affects Asset Validation in protected_deposit_allowed	Informational	Fixed
11	Precision Loss in Integer Division Leads to Smaller max_exposure	Informational	Fixed
12	Improper Account Used When Invoking Kamino Withdraw	Informational	Fixed
13	No Support for Multiple Pools in the Current UserAccount and UserReferrer	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Max Exposure Check May Fail Due to No Valid Deposit Solution

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `check_pool_protocol_weights` function, the `max_exposure` is calculated to ensure that the maximum exposure for each protocol is within limits. The formula for `max_exposure` is:

```
max_exposure = (regular_tvl * regular_max_exposure +
    protected_tvl * protected_max_exposure) / (regular_tvl +
    protected_tvl)
```

This formula combines two separate exposure limits for regular and protected deposits. However, the new exposure restriction assumes that if this formula holds, there will always be a valid deposit distribution satisfying both the regular exposure limit and the protected exposure limit.

A counterexample provided in the Proof of Concept section demonstrates that there are cases where no valid deposit distribution exists, even if the restriction is satisfied.

Impact

The inability to ensure a valid deposit distribution violates the core exposure invariant, potentially undermining the protocol's risk model. This could lead to excessive exposure to certain protocols and increase systemic risk.

Proof of Concept

Consider the following scenario:

- `pool.regular_max_exposure = 1000`
- `pool.protected_max_exposure = 4000`
- `pool.regular_amount = 150`
- `pool.protected_amount = 250`
- 4 protocols with:
 - `weights = [2500, 2500, 2500, 2500]`
 - `balance_native = [100, 100, 100, 100]`
- `pool.get_max_protocol_exposure() = 2875`

In this situation:

1. The `check_pool_protocol_weights` function passes.



2. Due to the `regular_max_exposure` limit, at most 15 amounts of the regular asset can be deposited into any single protocol. This means a total of 60 amounts of the regular asset are allowed to be deposited across all protocols.
3. However, the `balance_native` distribution shows that all 150 amounts of regular assets are deposited across the four protocols, which violates the `regular_max_exposure` limit.

Recommendation

Consider the following options:

- Maintain distinct deposit and exposure limits for regular (LUSD) and protected (PUSD) assets.
- Introduce a constraint such as `weight ≤ exposure` for each protocol.

Mitigation Review Log

Fixed in [PR1609](#).

4.2 Redundant Checks May Lead to Solend Withdrawal Failure

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In `SolendPoolWithdraw.cpi_withdraw`, the `collateral_token_account` is the pool's ATA for the `collateral_mint`, and the following check requires the program to ensure that the `collateral_token_account.amount` is 0 before withdrawing funds from Solend using `withdraw_obligation_collateral`.

```
435 // We start with 0 ctoken balance
436 let start_ctoken_balance = collateral_token_account.amount;
437
438 require_eq!(start_ctoken_balance, 0);
```

[programs/flexlend/src/protocols/pool/solend_pool.rs#L435-L435](#)

Currently, a malicious user who also has an ATA on the same `collateral_mint` can directly perform an SPL transfer of amount 1 to the `collateral_token_account`. This would make the `collateral_token_account.amount` greater than 0, causing the `require_eq!(start_ctoken_balance, 0)` check to fail.

Other checks with similar impacts include:



```
487     let last_ctoken_balance = collateral_token_account.amount;
488     // Assert that solend withdraw all tokens and did not hit any
489     // rate limits
489     require_eq!(last_ctoken_balance, 0);
```

[programs/flexlend/src/protocols/pool/solend_pool.rs#L487-L487](#)

Impact

Malicious users can directly perform an SPL transfer with an amount of 1 to the `collateral_token_account`. This prevents all assets deposited by the pool in Solend from being withdrawn through the `withdraw_protocol` IX. At this point, automation can only deposit into Solend but cannot withdraw.

Recommendation

It is recommended to remove the checks `require_eq!(start_ctoken_balance, 0)` and `require_eq!(last_ctoken_balance, 0)`. Since the withdraw amount is specified through the parameters of `cpi_withdraw` and already calculates the difference in `collateral_token_account.amount` before and after the invoke, it is not strictly required for `collateral_token_account.amount` to be 0.

Mitigation Review Log

Fixed in [PR1611](#).

4.3 Improper Obligation Initialization Check May Lead to Initialization Failures

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

In `SolendPoolInitialize.initialize_protocol_accounts`, it determines whether to initialize the obligation based on the data of the obligation account.

```
77     // obligation[18] is lending_market bytes
78     if obligation.try_borrow_data()?.[18] == 0 {
79         invoke_signed(
80             &init_obligation(
81                 ...
```

[programs/flexlend/src/protocols/pool/solend_pool.rs#L77-L81](#)



However, it uses a specific byte in the `lending_market` pubkey, which theoretically could contain a `\x00` byte in a pubkey on Solana.

As a result, it is possible that the single byte of the `obligation.lending_market` pubkey happens to be `\x00`, causing the same obligation account to be re-initialized even though it has already been initialized.

Impact

In Solend's `InitObligation` IX, there is a check to determine whether an obligation account has already been initialized. For an already initialized obligation account, an error will be returned directly. This could lead to the failure of the `SolendPoolInitialize.initialize_protocol_accounts`, preventing Solend from initializing on this obligation account.

Recommendation

Solend provides a method to determine whether an Obligation Account has been initialized, which can be done by checking `Obligation.version == UNINITIALIZED_VERSION` to determine whether the account needs to be initialized.

```
305 impl IsInitialized for Obligation {
306     fn is_initialized(&self) -> bool {
307         self.version != UNINITIALIZED_VERSION
308     }
309 }
```

[token-lending/sdk/src/state/obligation.rs#L305-L310](#)

It is recommended to refer to Solend's implementation and check whether `obligation.version` is `UNINITIALIZED_VERSION (0)`.

Mitigation Review Log

Fixed in [PR1612](#).

4.4 Inconsistent Pending Withdrawal Amount Counting for Weight Calculation

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `deposit_protocol` function, deposited weight is calculated to ensure the weight does not exceed the target ratio:



```
actual_weight = protocol_config.balance_native /  
    (allocation.regular_amount + allocation.protected_amount)
```

In the `check_pool_protocol_weights` function, the deposited weight is calculated to ensure the weight does not exceed the exposure ratio:

```
pool.available_liquidity = sum(allocation.protected_amount +  
    allocation.regular_amount - allocation.pending_withdrawals)  
actual_weight = sum(protocol_config.balance_native) /  
    pool.available_liquidity
```

The inconsistency arises because the weight formula in the `deposit_protocol` function includes pending withdrawal amounts in the denominator, while the `check_pool_protocol_weights` function omits pending withdrawal amounts in its denominator. However, in both cases, the numerator does include pending withdrawal amounts.

Impact

This leads to a mismatch in how weights are calculated, depending on whether pending withdrawal amounts are considered part of the liquidity pool. Specifically, the inconsistency could result in larger-than-expected weights during exposure checks when pending withdrawal amounts are treated as preserved.

Recommendation

Ensure that the pending withdrawal amounts are handled uniformly in both `deposit_protocol` and `check_pool_protocol_weights`.

Mitigation Review Log

Fixed in [PR1562](#).

4.5 Possible Precision Loss When Calculating Referral Fee Share

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Precision

Description

In the calculation of `calculate_referral_fee_share`, the `u64` values of `Allocation.total_referred_amount` and `Allocation.deposited_liquidity()` are directly converted into `f64`, and then a division operation is performed on the two `f64` values.

```
let share = self.total_referred_amount as f64 / deposited_amount as  
    f64;
```



[programs/flexlend/src/state/pool/liquidity.rs#L393-L393](#)

However, due to the limited precision of `f64` in Rust, there is a certain degree of precision loss when representing very large integers (exceeding 2^{53} , decimals: ~16).

Therefore, if `Allocation.total_referred_amount` and `Allocation.deposited_liquidity()` reach the magnitude of 2^{53} , there might be precision loss during the conversion from `u64` to `f64`, which could result in the calculated `referral_fee_share` being inaccurate.

Impact

When `u64` values are sufficiently large, the converted `f64` value may either increase or decrease, leading to an inaccurate calculation of `referral_fee_share`. Consequently, an imprecise referral fee, which could either be higher or lower than expected.

Proof of Concept

Below are two examples of `u64` to `f64` conversions.

```
fn main() {
    let x: u64 = 1801439850948198799;
    let y: f64 = x as f64;
    println!("larger: u64 {} => f64 {}", x, y);
    let a: u64 = 1801439850948198779;
    let b: f64 = a as f64;
    println!("smaller: u64 {} => f64 {}", a, b);
}
```

The output results demonstrating that one conversion increases the value, while the other decreases it.

```
larger: u64 1801439850948198799 => f64 1801439850948199000
smaller: u64 1801439850948198779 => f64 1801439850948198700
```

Recommendation

The calculation of `share` can be modified to use `u128` integer-only multiplication and division.

Mitigation Review Log

Fixed in [PR1630](#).

4.6 Precision in Referral Rewards Calculation May Leave Dust in UserReferrer

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Precision

Description

In `calculate_claimable_referral_rewards`, `claimable_amount` is calculated based on `claimable_c_amount` and `referral_exchange_rate`.

```
let claimable_amount =  
    referral_exchange_rate.fraction_collateral_to_liquidity(claimable_c_  
        amount);  
let claimable_c_amount: u64 = claimable_c_amount.to_floor();  
let claimable_amount: u64 = claimable_amount.to_floor();
```

[programs/flexlend/src/state/pool/referrals.rs#L46-L56](#)

Assume the values of these variables are as follows:

```
claimable_c_amount = 0.9  
refereral_exchange_rate = 0.9  
claimable_amount = claimable_c_amount / refereral_exchange_rate = 1.0
```

Then, by consecutively applying the `floor` operation to `claimable_c_amount` and `claimable_amount`, the results are as follows:

```
claimable_c_amount = 0  
claimable_amount = 1
```

This allows extracting referral fee with an amount of 1 without consuming any c tokens.

Impact

This may result in the User being able to extract the same amount of referral fee while consuming less LP Amount. Additionally, a small amount of dust might remain in the User Referrer, which can never be fully claimed. The User can repeatedly use this dust to extract small amounts of the input mint.

Recommendation

Performing the floor operation on `claimable_c_amount` before calculating `claimable_amount` can avoid the precision issues.

Mitigation Review Log

Fixed in [PR1627](#).



4.7 Slight Decrease of External Protocol TVL Can Cause Protocol DoS

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

Description

The `refresh_protocol_rewards` instruction calculates accrued interest using the current `balance_native` and the new `TVL`. If the new `TVL` decreases, the instruction fails and prevents continuous execution of user instructions after a time period.

The relevant code snippet is as follows:

```
63 // throw error if rewards is too negative, something is wrong
64 if accrued < -10 {
65     return Err(error!(Errors::InvalidRewardsRefresh));
66 }
```

[programs/flexlend/src/instructions/pool/refresh_protocol_rewards.rs#L63-L66](#)

This logic assumes that any negative accrued rewards larger than 10 amount represent an error. However, in scenarios where external protocols implement mechanisms like social loss (e.g., to cover bad debt), small and temporary decreases in `TVL` may occur. For instance, a decrease as minor as 0.1 basis points (bps) might self-correct within hours. Nonetheless, under the current implementation, such slight drops would cause the protocol to become unusable until the loss is fully covered.

Impact

Even a minimal decrease in `TVL` —which is not indicative of a hack or major issue—can cause the entire protocol to enter a denial-of-service (DoS) state. This disrupts operations and user experience unnecessarily, despite the fact that the `TVL` may recover automatically over time.

Recommendation

Introduce a new time-based tolerance threshold for slight percentage decreases in `TVL`.

Mitigation Review Log

Lulo Labs: This could be a good thing for us to track, it might be a small instance where there is bad debt.



4.8 Informational and Undetermined Issues

Missing security_check in pool_refresh_kamino_alt IX

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Data Validation

In the program, a `security_check` was added for many user and automation IXs in response to a security incident, but the `pool_refresh_kamino_alt` IX was omitted.

It is recommended to also add `#[access_control(security_check(&ctx.accounts.base_accounts.pool))]` to the `pool_refresh_kamino_alt` IX.

Missing Range Checks When Updating Pool Parameters

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Data Validation

In the Pool, `protocol_fee_bps`, `referral_fee_bps`, `protected_interest_share_bps` and `coverage_float_bps` are percentage-related configuration parameters and must not exceed 100% (10000).

However, in the related admin operations, there is no range check for the new parameter values, which might result in accidental updates setting these parameters to values greater than 10000.

- `update_protocol_fees` IX: Update `protocol_fee_bps` and `referral_fee_bps`.
- `update_protected_interest_share` IX: Update `protected_interest_share_bps`
- `update_coverage_float` IX: Update `coverage_float_bps`

It is recommended to add validity checks for the new values when using the above IXs to update parameters, to prevent these parameters from being set to invalid values.

Precision Loss Affects Asset Validation in protected_deposit_allowed

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Precision

In the implementation of the `protected_deposit_allowed` function, the calculation ensures that the current asset state of the pool satisfies the condition `LUSD × coverage_float ≥ PUSD / coverage_ratio`. Considering that all variables in this calculation are integers and involve integer division, there might be some precision loss due to integer division.

It is recommended to adjust the above calculation process by modifying the validation target from `LUSD × coverage_float ≥ PUSD / coverage_ratio` to `PUSD ≤ LUSD ×`



`coverage_ratio × coverage_float` . This does not change the mathematical inequality being validated but avoids precision errors caused by integer division.

Furthermore, when calculating `coverage_ratio`, integer division is also involved. If extremely high precision is required, the calculation expression for `coverage_ratio` can also be reconsidered to further reduce precision loss caused by division.

Precision Loss in Integer Division Leads to Smaller `max_exposure`

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Precision

In the calculation of `calculate_max_exposure` , integer division is performed twice.

The first occurs when using the function `calculate_weighted_liquidity` to compute `lUSD_weighted_balance` and `PUUSD_weighted_balance` .

```
92 liquidity
93     .checked_mul(weight.into())
94     .ok_or(Errors::OverflowError)?
95     .checked_div(BPS_SCALE.into())
96     .ok_or(error!(Errors::UnderflowError))
```

[programs/flexlend/src/utils/math.rs#L92-L96](#)

The second occurs during the final calculation of `max_exposure`.

```
129 let max_exposure: u32 = total_weighted_balance
130     .checked_mul(BPS_SCALE.into())
131     .ok_or(error!(Errors::OverflowError))?
132     .checked_div(total_balance)
133     .ok_or(error!(Errors::UnderflowError))?
134     .try_into()
135     .map_err(|_| error!(Errors::ConversionError));
```

[programs/flexlend/src/utils/math.rs#L129-L135](#)

To obtain `max_exposure` , the calculation involves three integer division operations in total. Since integer division introduces precision loss, using multiple integer divisions increases the cumulative precision loss. As a result, the calculated `max_exposure` might be smaller than expected.

Due to the multiple integer divisions in this calculation process, the final `max_exposure` could be smaller than the expected value. Although this deviation is minimal, a slightly smaller `max_exposure` might make an otherwise invalid protocol weight appear valid.

It is recommended to minimize the number of integer divisions in the calculation process. In this case, two of the division operations can be merged into one.



Improper Account Used When Invoking Kamino Withdraw

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Code QA

In `KaminoPoolWithdraw.cpi_withdraw`, when invoking `kamino_cpi::cpi::withdraw_obligation_collateral_and_redeem_reserve_collateral`, the

`placeholder_user_destination_collateral` is set to a liquidity account, whereas it should be a collateral account.

```
placeholder_user_destination_collateral: self
    .base_accounts
    .pool_input_token_account
    .to_account_info(),
```

[programs/flexlend/src/protocols/pool/kamino_pool.rs#L428-L431](#)

Although this is currently filled incorrectly, the account is not being used in Kamino at the moment, so it has no impact. However, in case Kamino updates in the future to utilize this account, it is recommended to fill it correctly now.

No Support for Multiple Pools in the Current UserAccount and UserReferrer

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Design

In the current `UserAccount` and `UserReferrer`, the scenario of having multiple pools is not considered, and all bookkeeping is targeted at a single pool.

```
20 pub struct UserAccount {
21     ...
22     // pool collaterals
23     pub regular_allocations: [u64; POOL_MAX_ALLOCATIONS],
24     pub protected_allocations: [u64; POOL_MAX_ALLOCATIONS],
```

[programs/flexlend/src/state/user.rs#L20-L22](#)

```
pub struct UserReferrer {
    pub owner: Pubkey,
    pub referred_amounts: [u64; POOL_MAX_ALLOCATIONS],
    pub referral_supplies: [u64; POOL_MAX_ALLOCATIONS],
    pub referral_fee_unclaimed: [u64; POOL_MAX_ALLOCATIONS],
```

[programs/flexlend/src/state/pool/referrals.rs#L7-L11](#)

The current implementation only allows the existence of one pool with a `pool_id` of `0`, which is temporarily safe.

However, in the future, if the design changes to allow the creation of more than one pool, the relevant code implementation of `UserAccount` and `UserReferrer` will need to be



redesigned. Otherwise, multiple pools sharing the same allocation bookkeeping could lead to bad debts.



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

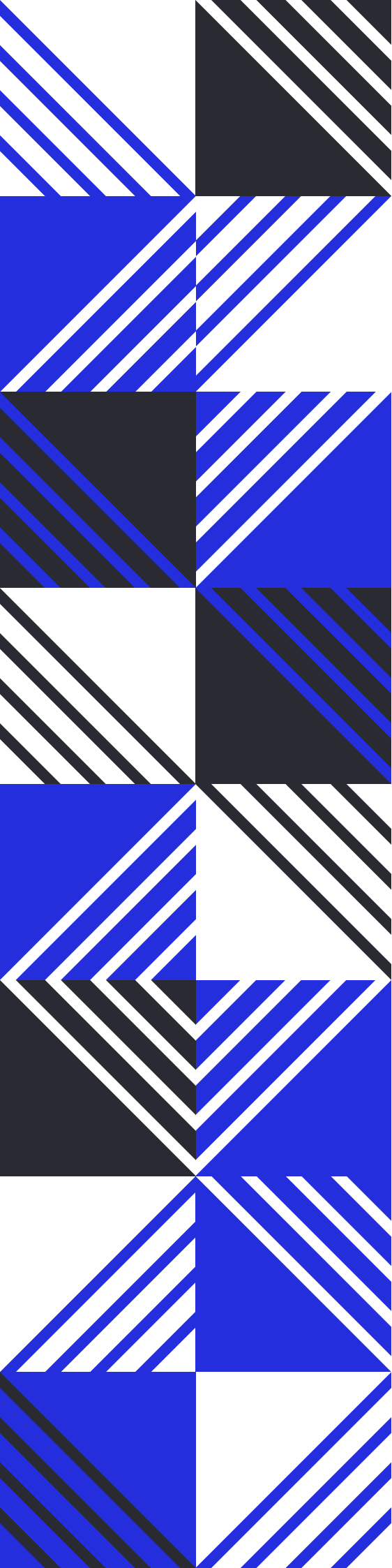
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs